

Feature: Site-wide SEO pass

From build-plan: feature 9

Branch: `feature/site-wide-seo-pass`

Goal

The work that genuinely needed every page to exist: a sitemap and a robots file covering both locales, `LocalBusiness` data for the business itself, one base URL that drives all of it, and a repeatable check that no page ever ships without its own metadata again.

It also settles a question nobody has asked out loud yet: **whether this site should be indexable at all before it has a domain**. The answer here is no, and that is enforced rather than remembered.

In scope

- One site URL, from an env var, driving `metadataBase`, the sitemap and robots.
- `app/sitemap.ts` generating both locales from the existing `routes` manifest.
- `app/robots.ts`, which refuses indexing until a real domain is configured.
- `metadataBase` on the root layout, so every canonical and hreflang the site already emits becomes absolute.
- `LocalBusiness` structured data on the home page.
- `scripts/auditSeo.mjs`: a repeatable check over the built output.

Out of scope

- **Registering the domain**. Not code. See Open questions.
- **Google Search Console and the Business Profile**. Both are external setup, both matter more than anything in this feature, and neither is a file in this repo.
- **Analytics**. The plan is explicit: no script, no cookie banner, and the contact form's "how did you find me" field plus Search Console is the measurement.
- **Changing any page's existing metadata**. Every page already sets its own `alternates`; this feature makes them absolute and proves they are there. If the audit finds one missing, that is a fix, not a rewrite.
- **Spanish keyword work**. Locked: Spanish is not an SEO target. The Spanish pages exist so a Spanish reader can use the site, and `hreflang` exists so Google does not treat the two as duplicates.
- **OpenGraph and Twitter cards**. Not in the build-plan line, and they need the share image nobody has made.

Build loop

`blueprint/config.json` sets `workflow.stepReview: "feature"` and `workflow.checkpointCommits: "disabled"`: steps in order, no commits during implementation, one review at the end. `/complete` makes the single commit.

No declared Verify command. The gate is `npm run build` and `npm run lint`, plus `node scripts/auditSeo.mjs` once step 5 exists. Baseline on `main` at spec time: build passes, fifteen routes.

Every step runs today. Nothing here is blocked. What is blocked is the *value* of it: without a domain the base URL is unset, and the robots file will correctly refuse indexing until that changes.

Build steps

- ☒ **1. One base URL, and `metadataBase`.** `lib/siteUrl.ts` resolving `NEXT_PUBLIC_SITE_URL` into a normalised origin, plus a flag for whether it was configured at all. Set `metadataBase` in the root layout from it. **Done when:** `npm run build` passes; with the variable set, every page's canonical and all three hreflang links are absolute and on that origin; with it unset the build still succeeds and the links stay relative rather than pointing at a wrong host.
- ☒ **2. The sitemap.** `app/sitemap.ts`, generated from `routes` in `lib/locale.ts` crossed with both locales, each entry carrying its `alternates.languages`. **Done when:** `/sitemap.xml` lists every route in both locales **except** `/thank-you`; every URL is absolute; each entry carries its `en`, `es` and `x-default` alternates; and the count matches `(routes.length - 1) * 2`.
- ☒ **3. Robots, and the indexing gate.** `app/robots.ts`. When the site URL is configured: allow everything except `/thank-you`, and point at the sitemap. When it is not: `Disallow: /` for every agent. **Done when:** with `NEXT_PUBLIC_SITE_URL` unset, `/robots.txt` disallows everything and names no sitemap; with it set, it allows crawling, disallows `/thank-you`, and includes the absolute sitemap URL.
- ☒ **4. `LocalBusiness` on the home page.** `localBusinessSchema` in `lib/schema.ts`, rendered once on the home page alongside the existing `FAQPage`. Shape in Data / contracts. **Done when:** `/` and `/es` each emit exactly one `HealthAndBeautyBusiness` block; it carries the name from `siteConfig`, Calgary as both `areaServed` and `addressLocality`, and **no** `streetAddress`; and no other page emits one.
- ☒ **5. The audit script.** `scripts/auditSeo.mjs`, run against the built output. **Done when:** `node scripts/auditSeo.mjs` exits 0 on the current build and exits non-zero naming the route when a canonical, an hreflang trio, a title, a description or the single `h1` is missing, or when a `noindex` route appears in the sitemap.
- ☒ **6. Run it, and fix what it finds.** **Done when:** the audit passes for all fifteen routes, `npm run build` and `npm run lint` pass clean, and anything the audit caught is either fixed here or written into the archive as a known gap.

Files / areas

New:

- `lib/siteUrl.ts`
- `app/sitemap.ts`
- `app/robots.ts`
- `scripts/auditSeo.mjs`

Changed:

- `app/[locale]/layout.tsx` - `metadataBase`
- `app/[locale]/page.tsx` - render `localBusinessSchema`
- `lib/schema.ts` - `localBusinessSchema`
- `.env.local` - `NEXT_PUBLIC_SITE_URL` . Untracked

Unchanged and load-bearing: `lib/locale.ts` already holds `routes` , the one list every page, rewrite and now sitemap is generated from. Adding a page still means adding one line there, which is the moment somebody notices it needs both languages.

Data / contracts

`NEXT_PUBLIC_SITE_URL` , an origin with no trailing slash and no path, for example `https://danimoreno.ca` . Public by design: it appears in every canonical.

`lib/siteUrl.ts` exports the normalised origin and a boolean for whether it was configured. That boolean is the indexing gate, and it exists so the decision is made in one place rather than remembered in three.

Unset is a first-class state, not an error. A local build and a preview deploy both run without it, and both must succeed. What they must *not* do is guess a host: a canonical pointing at the wrong origin is worse than a relative one, because Google believes it.

Sitemap entries. Absolute URLs, both locales, `alternates.languages` per entry with `en` , `es` and `x-default` . `/thank-you` is excluded because it is `noindex` : listing a page you have told Google not to index is a contradiction, and contradictions get resolved in ways nobody chose.

`lastModified` , `changeFrequency` and `priority` are all omitted. Google ignores the last two, and a `lastModified` that is really "whenever the build ran" is a lie told to a crawler on every deploy.

Robots.

| `NEXT_PUBLIC_SITE_URL` | `/robots.txt` | |---|---| | unset | `User-agent: *` / `Disallow: /` , no sitemap line || set | allow all, `Disallow: /thank-you` , absolute sitemap URL |

`LocalBusiness` , as `HealthAndBeautyBusiness` - a LocalBusiness subtype that says what she actually does:

```
{ "@context": "@type": "HealthAndBeautyBusiness",
  name, email, url,
  areaServed: { "@type": "City", name: "Calgary" },
  address: { "@type": "PostalAddress",
    addressLocality: "Calgary",
    addressRegion: "AB",
    addressCountry: "CA" },
  availableLanguage: ["en", "es"] }
```

No `streetAddress`, and that is a decision the site already made. The FAQ says "Calgary, by appointment. The exact address goes out when your booking is confirmed." Publishing one here would contradict the page and put a home address on the internet. The locality-level `PostalAddress` is what a service-area business can honestly say, and Google reads `areaServed` for the rest.

Omitted because they do not exist: `telephone`, `openingHours`, `geo`, `priceRange`, `logo`. Each would have to be invented, and an invented opening hour is a customer standing outside a door.

One `LocalBusiness` per site, on the home page only. Repeating it on every page does not strengthen it and gives Google several entities to reconcile.

The audit script reads the prerendered HTML under `.next/server/app`, strips `<script>` blocks first so the RSC payload cannot be mistaken for markup, and asserts per route: exactly one canonical, three `hreflang` links, a non-empty title and description, exactly one `<h1>`. It then cross-checks the sitemap against the route manifest and against which pages declare `noindex`. It exits non-zero naming the route and the failed assertion.

A script rather than a test because there is no test runner, and a standalone script is the shape `scripts/uploadImagesToImageKit.mjs` already established for this project.

Testing

No unit tests: no runner is configured. The audit script is the closest thing this feature has to one, and it is deliberately a check over real built output rather than a unit test of a helper.

No browser harness. `robots.txt` and `sitemap.xml` are checked by reading the built output and, if a server is running, by fetching them.

Both locales, as always. This feature is the one that would notice if a page had been shipping without its Spanish twin.

Notes for the AI

Generate from `routes`, never from a second list. `lib/locale.ts` is the manifest that already drives `generateStaticParams` and the rewrites. A sitemap with its own hand-written list is a sitemap that goes

stale the first time somebody adds a page.

Relative beats wrong. Where the base URL is unconfigured, leave links relative. Do not fall back to a Vercel host: an absolute canonical on a preview domain invites Google to index the preview as the real site, which is an own-goal that takes redirects to undo.

No component contains the word Dani. The schema reads `siteConfig`.

Do not invent business facts. No phone, no hours, no coordinates, no price range. They are absent from `siteConfig` because nobody has supplied them.

Animation is CSS. Do not add a package.

Open questions

1. There is still no domain, and it is now the last real blocker. The overview has carried this TODO since the planning baseline. It gates `metadataBase`, absolute `hreflang`, a useful sitemap, the Resend sending domain from item 7, and the Google Business Profile - which the overview says matters more than anything on this site for the first year.

This feature is built so that registering it and setting one variable turns everything on at once. Until then the site correctly refuses to be indexed.

2. Confirm: the site should not be indexable before the domain exists. Spec'd as `Disallow: /` whenever `NEXT_PUBLIC_SITE_URL` is unset, which is the state today. The reasoning: a `.vercel.app` host that gets indexed becomes the canonical version of this business in Google's eyes, and moving off it later costs redirects and ranking. The cost of the rule is that nothing is discoverable until the domain lands - which is already true, since nothing links here.

Say so now if you would rather the preview were crawlable.

3. The `[confirm]` on location, which the overview flags as "worth thinking about rather than defaulting". Spec'd as a service-area business: locality-level address, no street. That agrees with the FAQ copy she already approved. The alternative - publishing a street address - would change the schema, the FAQ answer, and what goes on the Business Profile, and it is a personal-safety decision rather than a technical one. It is hers, not ours.

4. `siteConfig.business.email` is still the agency's address, and this feature puts it into structured data as the business's public contact address rather than only inside a Server Action. That is wider exposure than item 7 gave it. It stays correct only until Dani has her own.

Status: verified in both configurations

`npm run build` exit 0, `npm run lint` exit 0, `node scripts/auditSeo.mjs` exit 0 across all 16 prerendered routes.

The feature has two states and both were exercised by actually setting the variable and rebuilding, rather than reasoned about:

```
|| NEXT_PUBLIC_SITE_URL unset | set to https://danimoreno.ca | |---|---|---| | robots.txt | User-Agent: * / Disallow: / , no sitemap line | Allow: / , Disallow: /thank-you , Disallow: /es/thank-you , sitemap URL || sitemap.xml | 0 URLs | 14 URLs, 42 hreflang entries || Canonical on /gallery | /gallery | https://danimoreno.ca/gallery || Canonical on /es/nails | /es/nails | https://danimoreno.ca/es/nails || Canonical on / | / | https://danimoreno.ca |
```

The sitemap's 14 URLs are 7 routes across 2 locales: `/thank-you` is excluded, confirmed by checking that no URL in the output contains it.

`HealthAndBeautyBusiness` renders on `en.html` and `es.html` and on no other page, alongside the `FAQPage` that was already there. It carries no `streetAddress` and no `url` , both verified by parsing the emitted JSON-LD.

The variable was removed again afterwards. The site ships refusing to be indexed, which is the correct state until a domain exists.

The audit script found two bugs, both in itself

This is the part worth reading.

It passed a page whose canonical pointed at the home page. The first version asserted that a canonical *existed*, not that it was *correct*. Proved by deleting a page's `alternates` block: the page then inherits the layout's, still has exactly one canonical and a complete `hreflang` trio, and the audit reported PASSED. That is precisely the failure this feature exists to prevent, sailing through the check written to prevent it. Fixed by comparing the emitted canonical against the route's own expected path. Re-proved by pointing `/about` at `/`:

FAILED (2):

`/en/about: canonical is "/", expected "/about"`

`/es/about: canonical is "/es", expected "/es/about"`

It reported "PASSED - 0 routes" on an empty build. The first deliberate break left unused imports, so the build failed, `.next/server/app` was empty, and the script found nothing to check and said so happily. A check that succeeds when there is nothing to check is worse than no check, because it gets trusted. Zero routes is now a failure.

Both are written into the script's own comments, next to the code they explain.

Departures from the spec

1. **The sitemap returns an empty array rather than preview URLs** when no site URL is configured. The spec said URLs must be absolute and left the unconfigured case to "the robots file refuses indexing"; returning nothing is the honest completion of that. An empty sitemap says "no pages to offer"; one full of `.vercel.app` URLs is a request to index the wrong site. 2. **`robots.ts` disallows `/es/thank-you` as well as `/thank-you`**. The spec named one path. They are the same page in two languages and a crawler does not know that. 3. **The audit gained a canonical-correctness assertion and a zero-route guard**, neither of which the spec listed, because without them it did not do the job the spec asked for.

Not verified

- **No rendered page was opened in a browser.** This feature emits no UI. What it emits was read out of the built output directly, which is what a crawler receives.
- **`robots.txt` and `sitemap.xml` were not fetched over HTTP.** They were read from `.next/server/app/*.body`, which is the same bytes Next serves.
- **Nothing has been submitted to Google.** Search Console and the Business Profile are external setup and out of scope, and the overview says the Business Profile matters more than anything on this site for the first year.
- **`siteConfig.business.email` is the agency's address** and now appears in structured data as the business's public contact address, which is wider exposure than the contact form gave it. Harmless while nothing is indexed.
- **The `[confirm]` on location stands.** The service-area shape agrees with the FAQ copy she approved, but publishing or withholding an address is her decision, not one made here.

Implementation walkthrough

One variable, three consumers

`lib/siteUrl.ts` exists so a single decision is made once rather than remembered in three files. It normalises `NEXT_PUBLIC_SITE_URL` to an origin - stripping a trailing slash, discarding any path, because a path in that value would be prefixed onto every canonical on the site - and exports whether it was configured at all.

That boolean is the indexing gate. `metadataBase`, `sitemap.ts` and `robots.ts` all read from this one module, so they cannot disagree about whether the site has a home.

A malformed value logs and is treated as unset rather than thrown on. A deploy that refuses indexing is recoverable; one that will not build is a worse Saturday.

Why relative beats wrong

The tempting shortcut is to fall back to the Vercel preview host so the sitemap has something absolute to say. That is an own-goal: if Google indexes the preview, the preview becomes the canonical version of this business, and moving to the real domain later costs redirects and whatever ranking accumulated on the wrong host.

A relative canonical resolves against whatever host served the page - wrong-but-harmless. An absolute one pointing somewhere else is believed. So `metadataBase` is `undefined` while unconfigured, every page's existing `alternates` stay relative, the sitemap is empty, and `robots.txt` says `Disallow: /`.

Generated from one list

`app/sitemap.ts` reads `routes` from `lib/locale.ts` - the same manifest that already drives `generateStaticParams` and the rewrites in `next.config.ts`. Adding a page stays one line in one file, which is also the moment somebody notices it needs both languages.

`lastModified`, `changeFrequency` and `priority` are all absent. Google has said for years it ignores the last two, and a `lastModified` that really means "whenever the build ran" is a lie told to a crawler on every deploy - it trains Google to distrust a field the site might one day need honestly.

The business, once

`localBusinessSchema` emits `HealthAndBeautyBusiness`, a `LocalBusiness` subtype that says what she actually does. More specific is free information.

It carries no `streetAddress`, and that is the site agreeing with itself: the FAQ already says "Calgary, by appointment. The exact address goes out when your booking is confirmed." Publishing one here would contradict the page and put a home address on the internet. A locality-level `PostalAddress` is what a service-area business can honestly say, and `areaServed` carries the rest.

`telephone`, `openingHours`, `geo`, `priceRange` and `logo` are all absent because they do not exist in `siteConfig`. Each would have to be invented, and an invented opening hour is a customer standing outside a door.

It renders on the home page only. Repeating it per page does not strengthen it and gives Google several entities to reconcile.

The audit

`scripts/auditSeo.mjs` reads the prerendered HTML rather than the source, because what matters is what a crawler receives. It strips `<script>` blocks first: the RSC payload contains serialised copies of every prop including the metadata, and matching against it would let an empty `<head>` pass.

Per route it asserts one canonical **pointing at that route**, a complete `hreflang` trio, a non-empty title and description, exactly one `h1`, and that `noindex` is present exactly where it should be. It then checks the sitemap for absolute URLs and for any `noindex` page that leaked in.

It is a script rather than a test because there is no runner, and `scripts/uploadImagesToImageKit.mjs` already established that shape here. It runs after a build and exits non-zero naming the route and the assertion.